Depth First Search

```
def dfs(start, is_goal, successors):
    """Generic DFS: works for ANY problem given a goal test and a successor fn."""
    stack = [(start, [start])]
    visited = {start}
    while stack:
        state, path = stack.pop()
        if is_goal(state):
            return path
        for nxt in successors(state):
            if nxt not in visited:
                visited.add(nxt)
                stack.append((nxt, path + [nxt]))
    return None


def water_jug_successors(state, cap_a=4, cap_b=3):
    a, b = state
    return [
        (cap_a, b), (a, cap_b), (0, b), (a, 0),
        (a - min(a, cap_b - b), b + min(a, cap_b - b)),   # pour A -> B
        (a + min(b, cap_a - a), b - min(b, cap_a - a)),   # pour B -> A
    ]


def program2():
    print("\n=== 2. DEPTH FIRST SEARCH (Water Jug 4L & 3L, get 2L in jug A) ===")
    path = dfs((0, 0), lambda s: s[0] == 2, water_jug_successors)
    for step in path:
        print(step)

    print("\nDFS on a graph (A -> G):")
    graph = {"A": ["B", "C"], "B": ["D", "E"], "C": ["F"], "D": [], "E": ["G"], "F": [], "G":
        []}
    print(" -> ".join(dfs("A", lambda s: s == "G", lambda s: graph[s])))


if __name__ == "__main__":
    program2()
```